Review 2: Methodology & Implementation Details

Course: BCSE303L Operating Systems | Faculty: Dr. V. Ezhil Arasi
Team: Nitin Sunil Holkar (24BCE2819), Team Member 2, Team Member 3
School of Computer Science and Engineering, Vellore Institute of Technology, Vellore

1. Process Lifecycle & State Machine

Patients progress through five formal states: NEW (registered, priority classified), READY (queued in scheduler, accumulating aging ticks), RUNNING (allocated resources, undergoing active medical treatment), WAITING (blocked due to resource contention or Banker's safety check), and COMPLETED (treatment finished, resources returned).

Preemption allows a running process to be moved back to READY if a higher-urgency Level 1 patient arrives. Contested resources are reclaimed, and the preempted task retains its executed burst progress.

2. Mathematical and Algorithmic Formulations

• Dynamic Aging: Effective Priority Peff(t) = max(2, Pbase - floor(t_ready / Ta)), with Ta = 4 ticks. Capping at Level 2 prevents starvation of lower urgencies while ensuring Level 1 Critical cases remain strictly dominant.

• Banker's Avoidance: Maintained via Available vector, Max matrix, Allocation matrix, and Need = Max - Allocation. Before granting any request Req <= Need, safety state is verified using Work and Finish vectors.

• Deadlock Detection (WFG): Directed edge Pi -> Pj forms if Pi is waiting for resource R held by Pj when Available[R] == 0. A recursive DFS flags cycles.

• Preemption Recovery: The cycle victim is chosen by min(-Peff, Progress, -ArrivalTime). The victim is preempted to READY, releasing resources to clear circular wait.

3. Technology Stack & Software Architecture

Layer	Technology	Description
Core Engine	Python 3.13 (threading, queue, lock)	Simulates concurrent patient processes and synchronization
Synchronization	Counting Semaphore & Mutex	Protects 5 resource pools and guarantees atomic allocation
Persistence	SQLite 3	Audits simulation runs, patient telemetry, and deadlock logs
Web Server	Python http.server (Port 5050)	Zero-dependency RESTful HTTP API and static file server
Dashboard UI	HTML5, CSS3, ES6 JavaScript	Aesthetic dark-mode clinical command center with live Kanban
Visual Graph	SVG Vector Canvas	Interactive Wait-For Graph with dynamic node layout and cycle arrows

4. Initial Test Workload (Proposal Validation)

Patient	Arrival	Priority	Burst	Resource Demands
P1	0	3 (Moderate)	8	1 Bed, 1 Doctor, 2 Nurses
P2	1	2 (Serious)	6	1 Bed, 1 Ventilator, 1 Doctor
P3	2	4 (Stable)	4	1 Doctor, 1 Nurse (Round Robin, Aging eligible)

P4	3	1 (Critical)	5	1 Bed, 1 Ventilator, 2 Doctors, 2 Nurses, 1 Theatre
P5	4	3 (Moderate)	7	1 Bed, 1 Doctor, 1 Nurse
P6	6	1 (Critical)	4	1 Bed, 1 Ventilator, 1 Doctor, 1 Theatre